Unit-4

Tree

Unit-4

- Basic Terminologies
- M-ary Tree
- Binary Tree
  - Array representation
  - Linked representation
- Binary Tree Traversals/Operations
- Conversion of General Tree to Binary Tree
- Binary Search Tree

Basic Terminologies

❖ Tree :

- A tree consists of a finite set of elements, called nodes, and a finite set of directed lines called branches, that connect nodes.

❖ Degree :

- The no. of branches associated with a node is the degree of the node.
- When the branch is directed toward the node, it is an in-degree branch ; when the branch is directed away from the node, it is an out-degree branch.
- The sum of in-degree and out-degree braches is the degree of the node.

Basic Terminologies

❖ In-degree :

- In-degree of a node is the number of edges arriving at that node.

❖ Out-degree :

- Out-degree of a node is the number of edges leaving that node.

Basic Terminologies

❖ Root :

- If the tree is not empty, then the first node is called the root.
- The in-degree of the root is by definition, zero.

❖ Leaf :

- A leaf is any node with an out-degree of zero, i.e, the node without a children.

❖ Internal node :

- A node that is not a root or a leaf is known as internal node because it is found in the middle portion of a tree.

Basic Terminologies

❖ Parent :

- A node is a parent if it has successor nodes - that is, if it has an out-degree greater than zero.

❖ Child :

- A node with a predecessor is a child. A child node has an in-degree of one.

❖ Siblings :

- Two or more nodes with the same parent are siblings.

Basic Terminologies

❖ Level :

- The level of a node is its distance from the root. Because the root has a zero distance from itself, the root is at level zero.

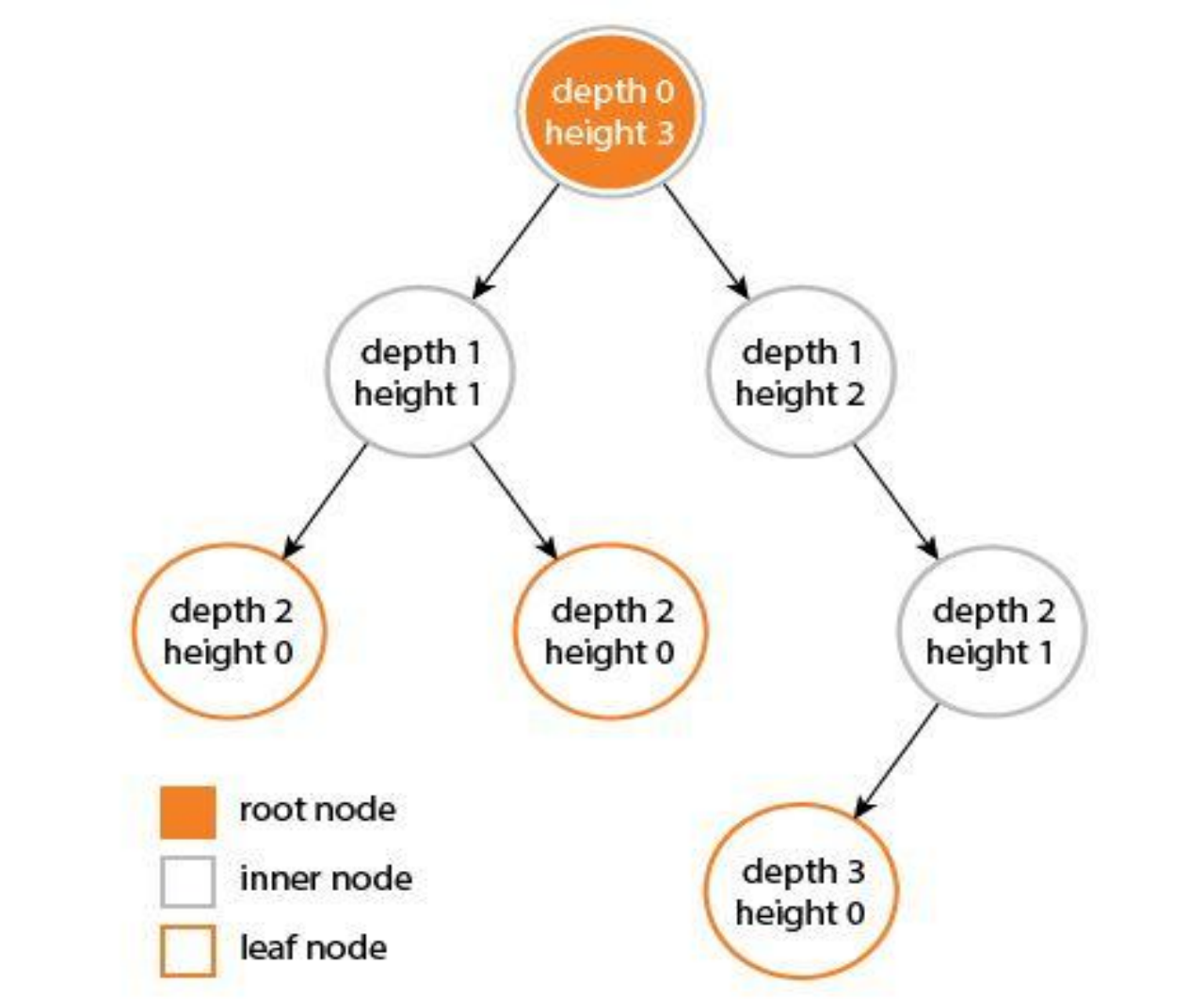
❖ Height:

- The height of a node is the length of the longest downward path to a leaf node from that node.
- The height of the root node is the height of the tree.

❖ Depth :

- Tree is drawn upside down, so no. of levels can be referred as depth of the tree.

Basic Terminologies



Basic Terminologies

❖ Ancestor :

- An ancestor is any node in the path from root to the node.

❖ Descendent :

- A descendent is any node in the path below the parent node. i.e. all nodes in the path from a given node to a leaf node are descendents of the node.

❖ Node :

- Each element in a tree is known as node of a tree.

Basic Terminologies

❖ Directed Tree :

- It is an acyclic digraph which has one node called its root, with in-degree 0, while all other nodes have in-degree 1.

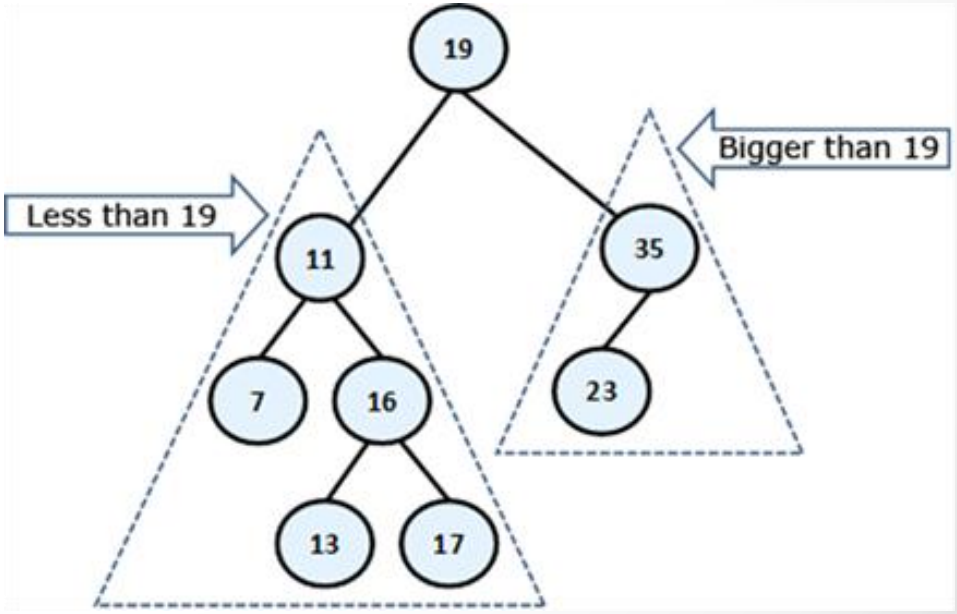
❖ Terminal Node and Branch Node :

- In a directed tree, any node which has out-degree 0 is called terminal node or leaf; all other nodes are known as branch nodes.

Basic Terminologies

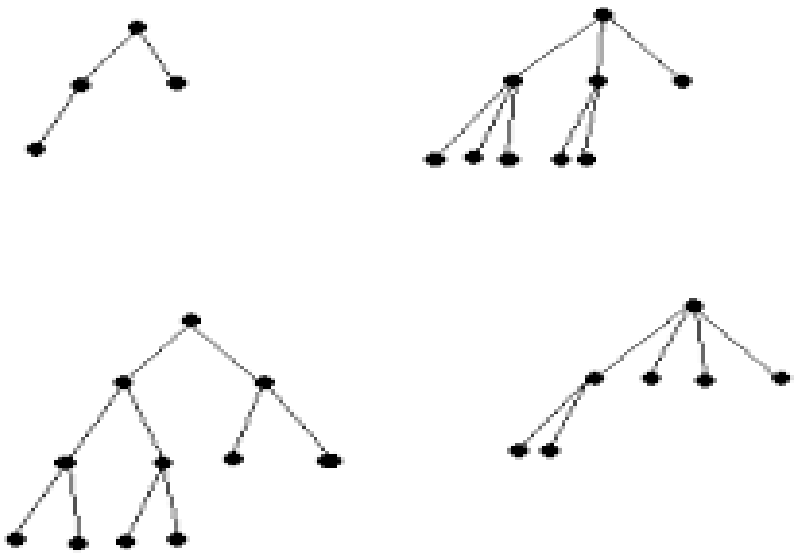
❖ Ordered Tree:

- An ordered tree contains nodes which can be ordered according to specific criteria.
- **Often it is a binary tree**, i.e. nodes have at most two children called left and right child. **The tree is ordered when at every node, all elements in its left child tree are smaller than elements in its right child tree.**



M-ary Tree

- A tree is called an M-ary tree if every internal node has no more than M children.
- The tree is called **full M-ary tree** if every internal node has exactly M children.
- An M-ary tree with M=2 is called a binary tree.



Binary Tree

- A Binary tree is a special case of tree in which no node of a tree can have more than two nodes.

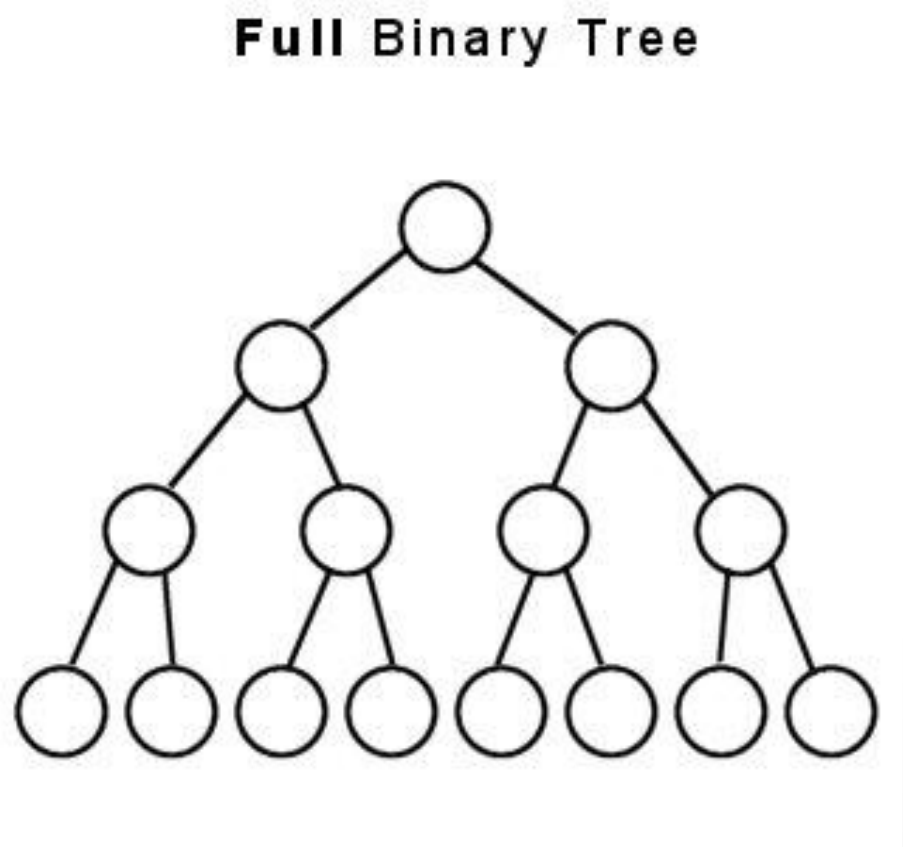
- For a binary tree, it has been found that:

- The maximum number of nodes at level i will be 2^i .
- If d is the depth of the tree then the maximum number of nodes that the tree can have is $2^{d+1}-1$.

Binary Tree

❖ Full Binary Tree:

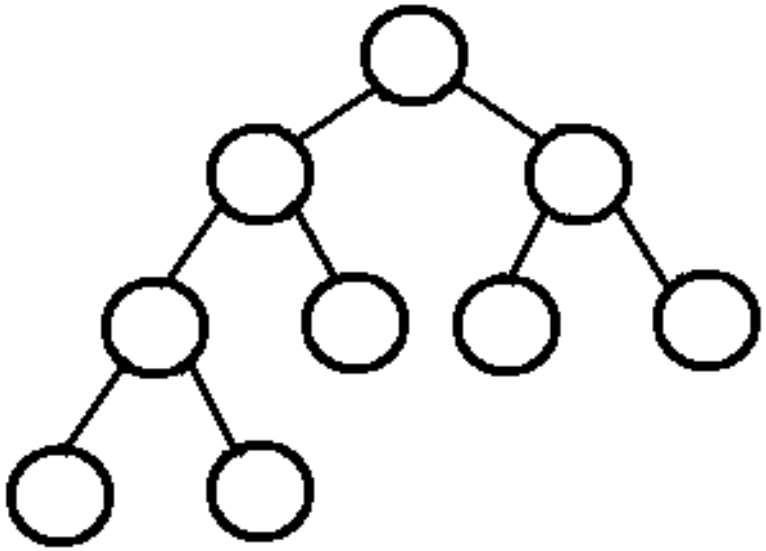
- A binary tree in which every node other than leaf nodes has two children.



Binary Tree

❖ Complete Binary Tree:

- A binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible.

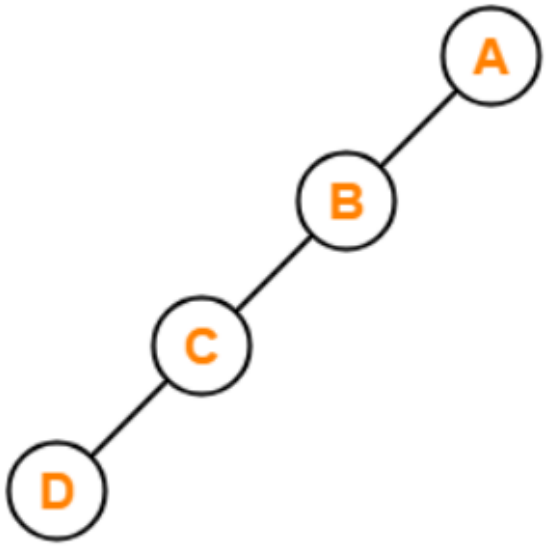


complete tree

Binary Tree

❖ Left Skewed Binary Tree:

- A binary tree in which in every node **right sub-tree is missing**, is known as Left Skewed Binary Tree.

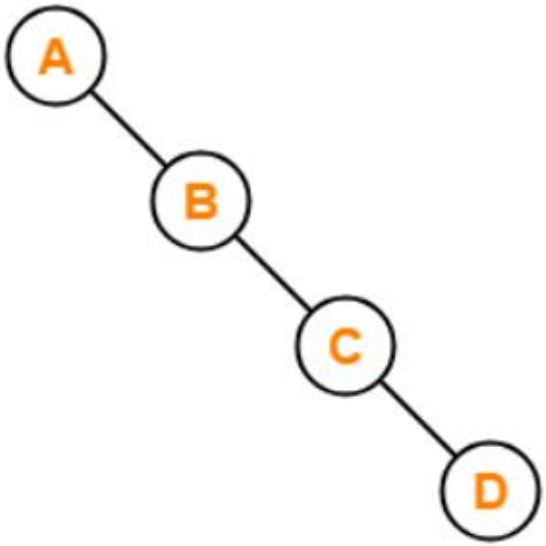


Left Skewed Binary Tree

Binary Tree

❖ Right Skewed Binary Tree:

- A binary tree in which in every node **left sub-tree is missing**, is known as Right Skewed Binary Tree.



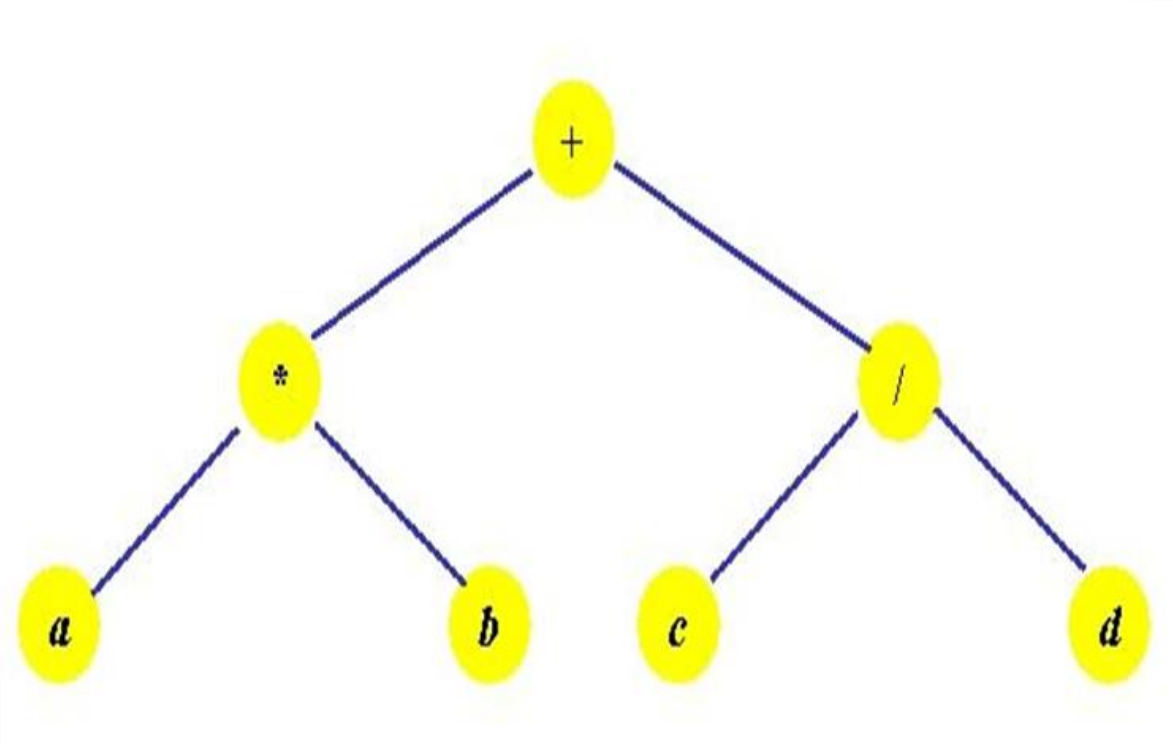
Right Skewed Binary Tree

Expression Tree

- An **expression tree** is a binary tree with the following properties:
  - Each leaf is an operand.
  - The root and internal nodes are operators.
  - Sub-trees are sub-expressions, with the root being an operator.

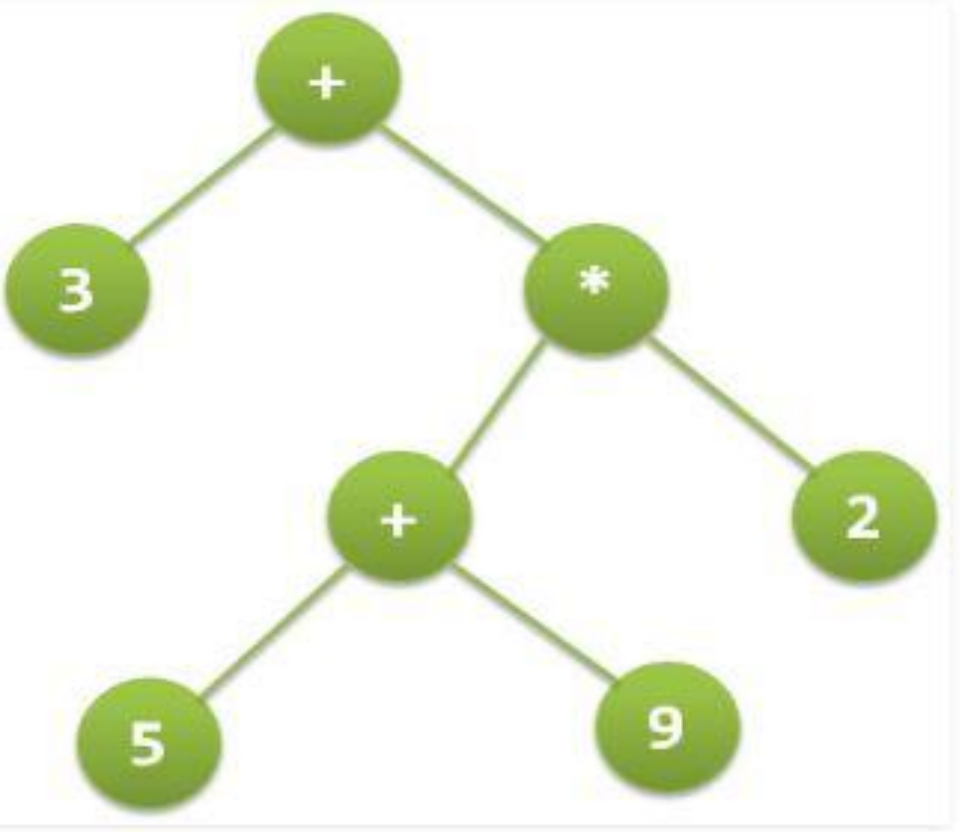
Expression Tree

- Expression : `(a * b) + (c / d)`



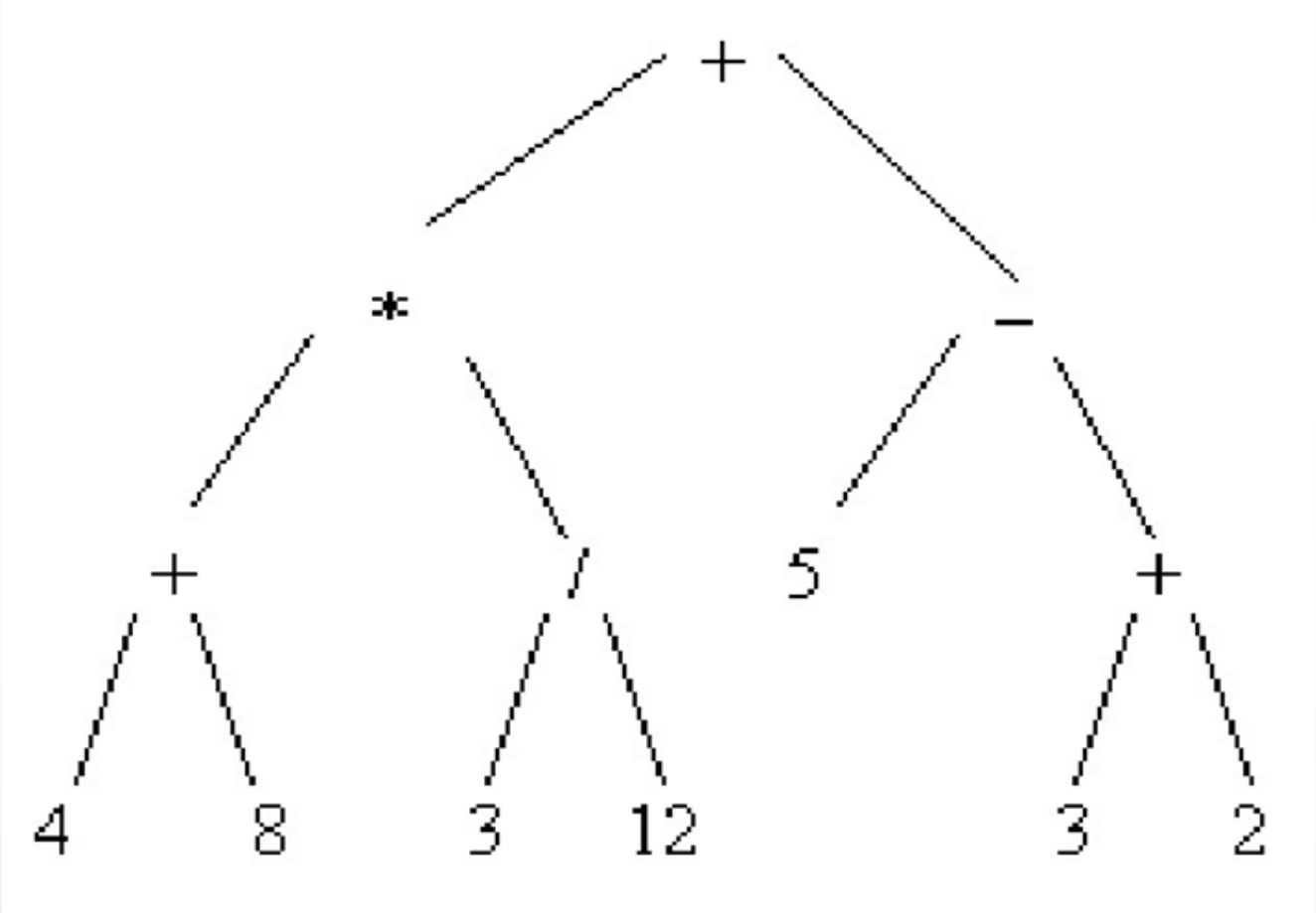
Expression Tree

- Expression : **3 + ((5+9)*2)**



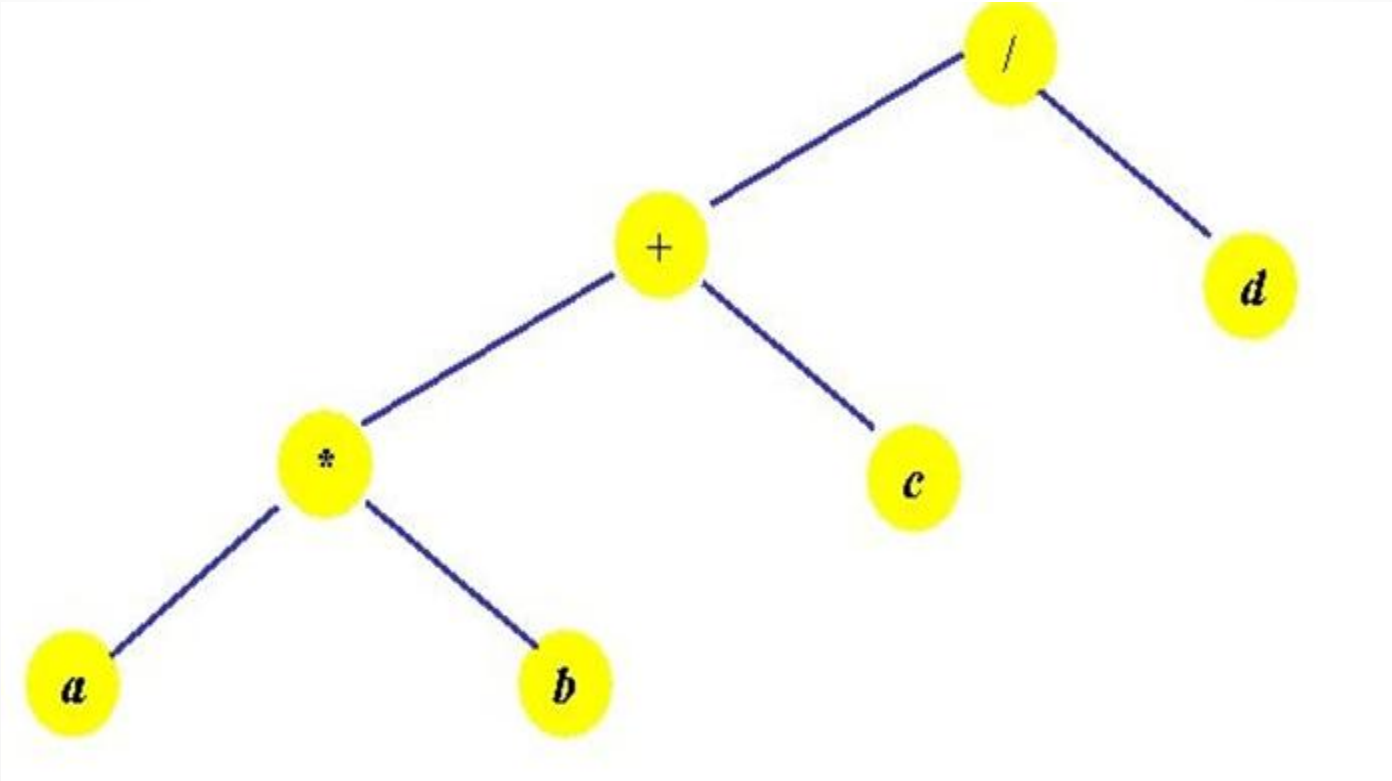
Expression Tree

- Expression : **(4+8)*(3/12)+(5-(3+2))**



Expression Tree

- Expression : `((a * b) + c) / d)`

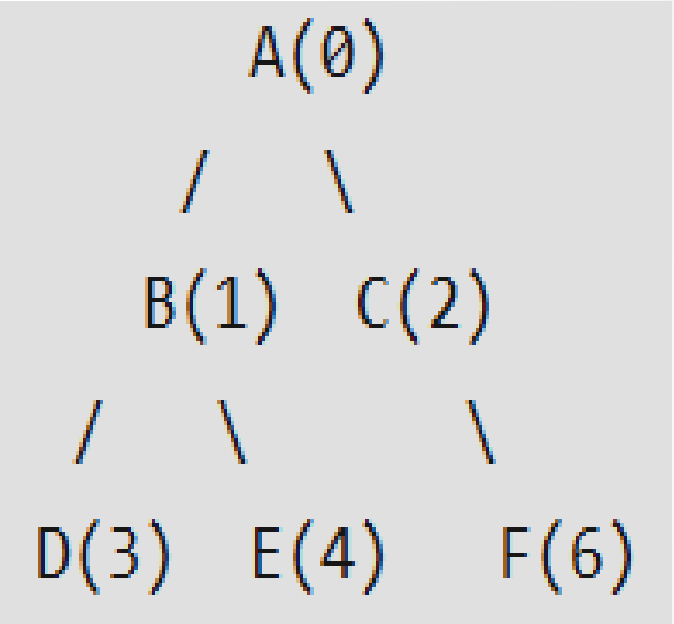


Storage Representation of a Binary Tree

- Trees can be represented in two ways:
  1. Array Representation (Static)
  2. Linked Representation (Dynamic)

Array Representation

- **The maximum size of an array is $2^{d+1}-1$, where d is the depth of the tree.**



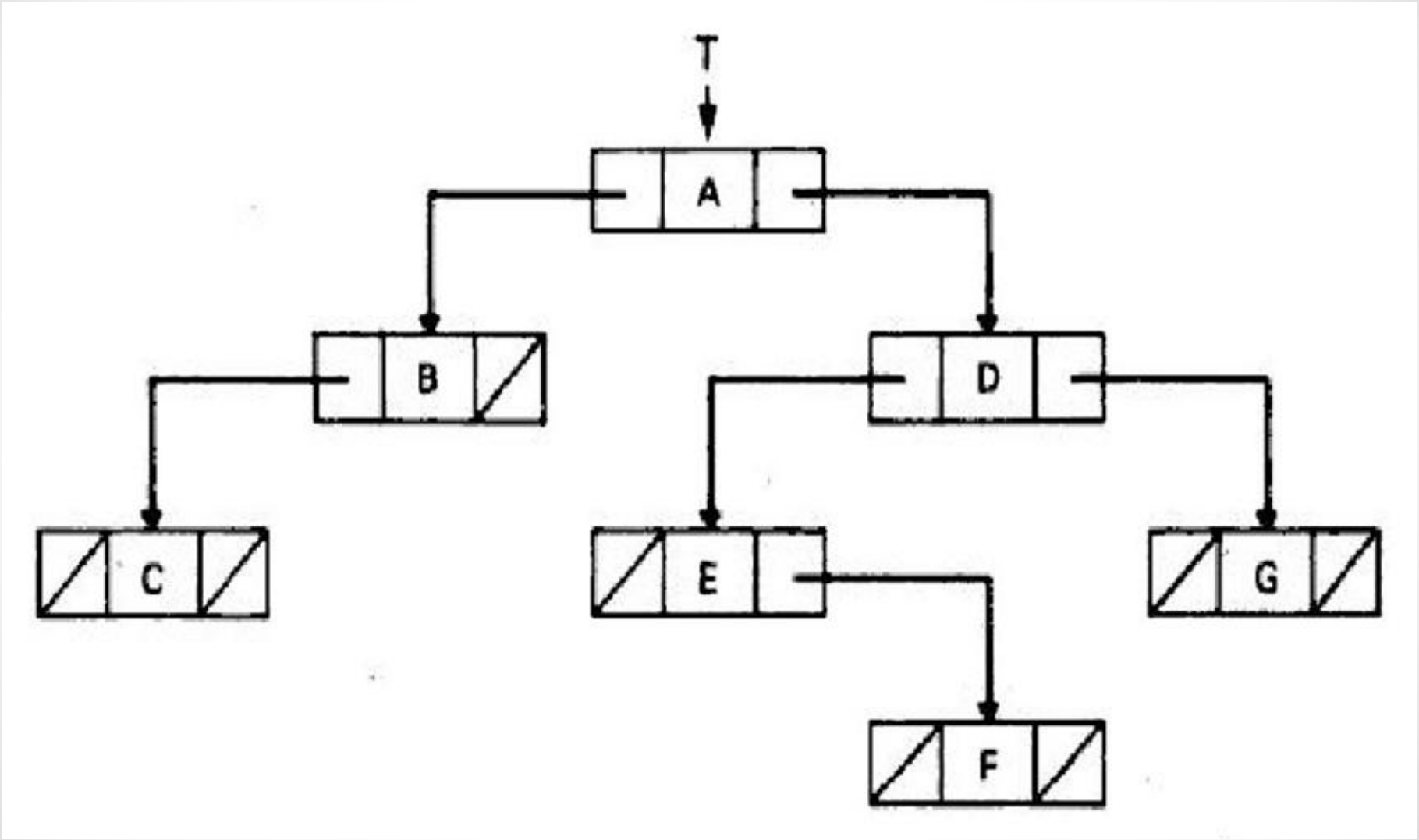
- If Parent = p, then
- left_child = (2*p)+1
- right_child = (2*p)+2
- Where, parent, left_child and right_child are the values of indices of the array.

Linked Representation

- In linked representation of a binary tree, every node will have three parts:
  1. A data element
  2. A pointer to the left node
  3. A pointer to the right node
- So in C, the binary tree is built with a node type given as follows:

```
struct node
{
        struct node *left;
        int data;
        struct node *right;
}
```

Linked Representation



Traversal of Binary Trees

- **Traversing a binary tree** is the process in which **each node in the tree should be visited exactly once**, in a systematic way.

- Unlike linear data structures in which the elements are traversed sequentially, a tree is a non-linear data structure in which the elements can be traversed in many different ways.

- There are many different ways for tree traversals.

- These ways differ in the order in which the nodes are visited.

Traversal of Binary Trees

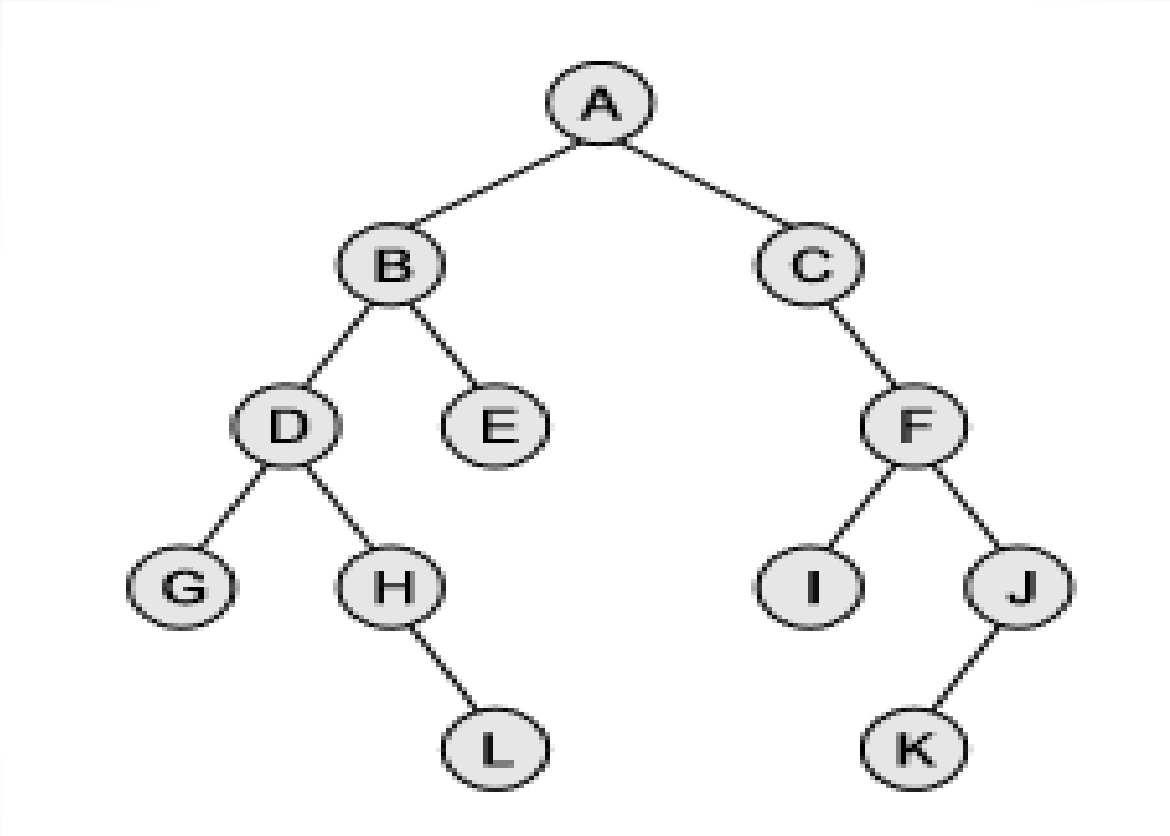
- Following are the different ways in which a binary tree can be traversed:
  - **Preorder Traversal**
  - **Inorder Traversal**
  - **Postorder Traversal**

Preorder Traversal

- To traverse a non-empty binary tree in preorder, the following operations are performed recursively at each node.

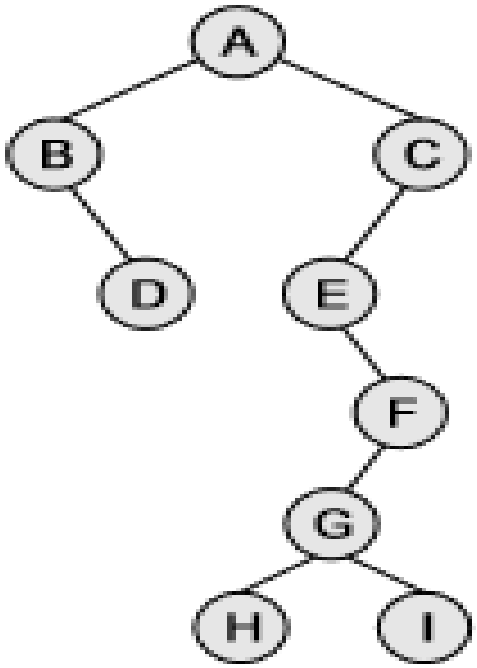
- The traversal starts with the root node of the tree and continues by:
  1. Visiting the root node.
  2. Traversing the left sub-tree.
  3. Traversing the right sub-tree.

Preorder Traversal



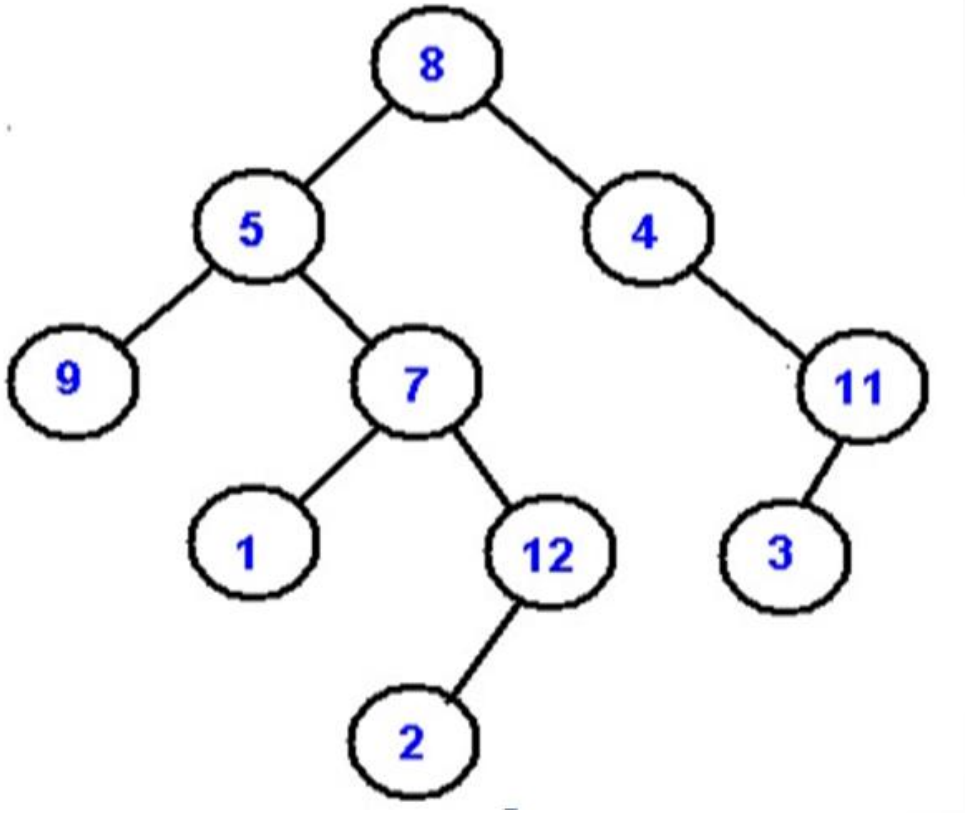
Pre-Order Traversal: A,B,D,G,H,L,E,C,F,I,J,K

Preorder Traversal



Pre-Order Traversal: A,B,D,C,E,F,G,H,I

Preorder Traversal



Pre-Order Traversal: 8, 5, 9, 7, 1, 12, 2, 4, 11, 3

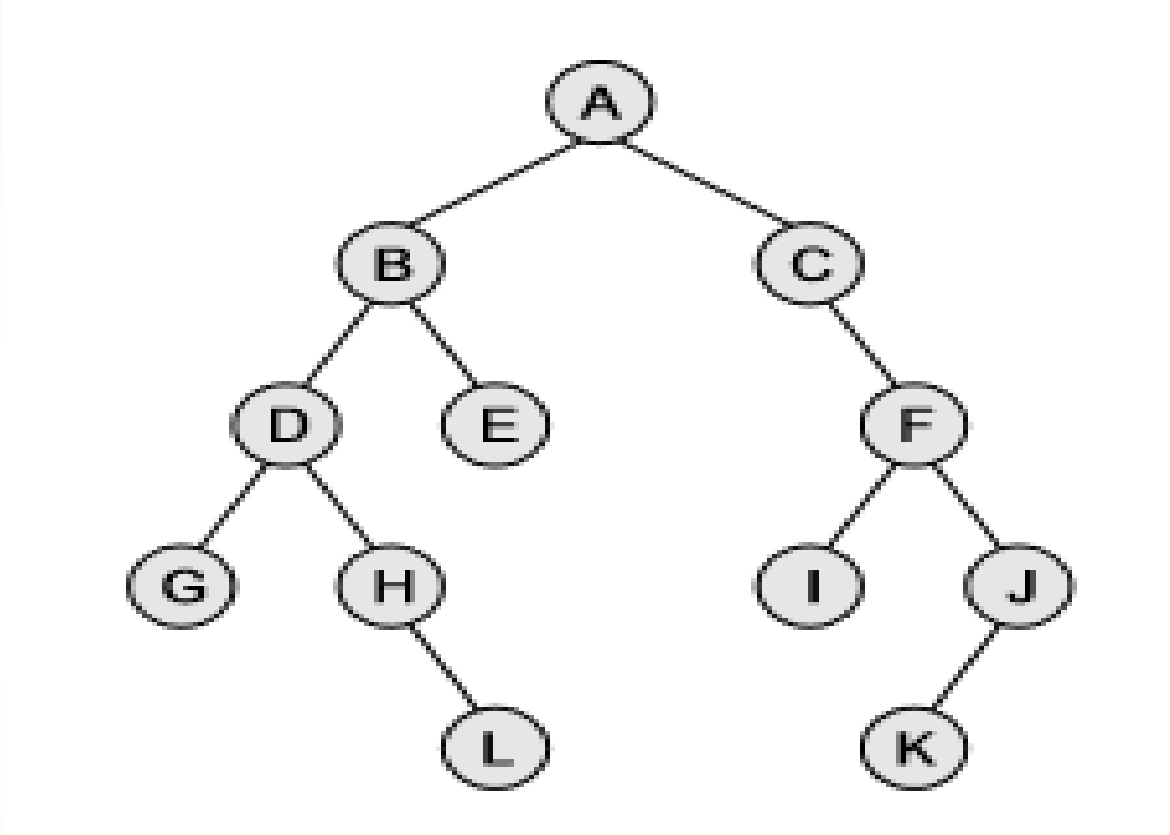
Inorder Traversal

- To traverse a non-empty binary tree in inorder, the following operations are performed recursively at each node.

- The traversal starts with the root node of the tree and continues by:

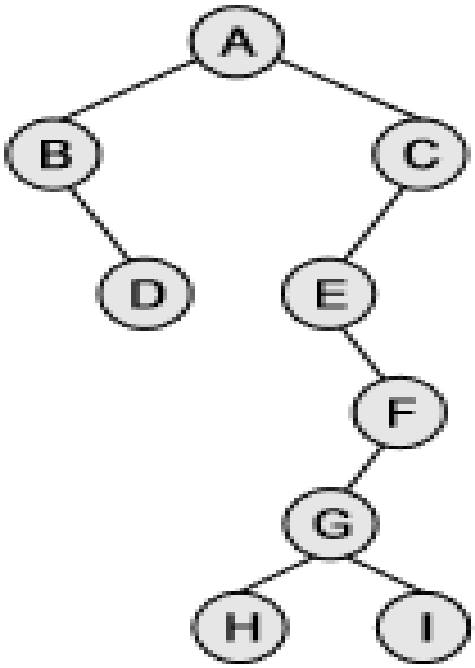
  1. Traversing the left sub-tree.
  2. Visiting the root node.
  3. Traversing the right sub-tree.

Inorder Traversal



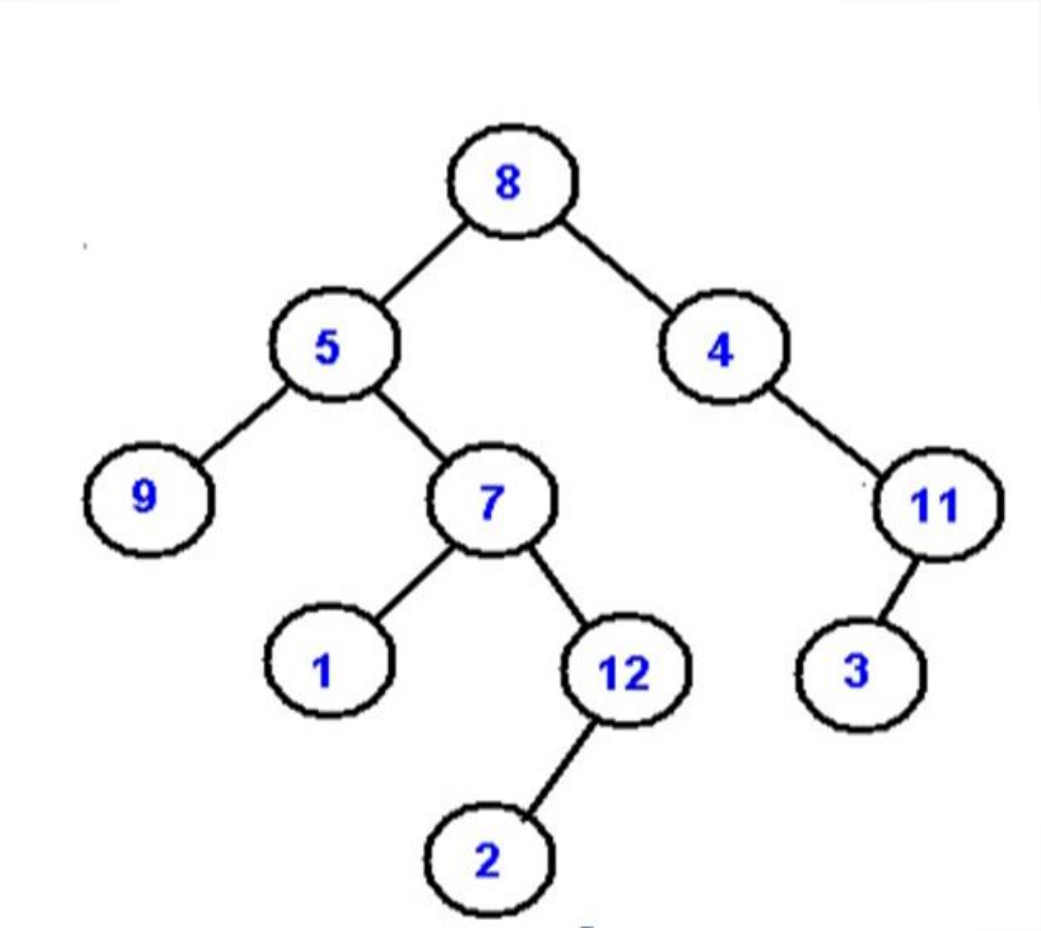
In-Order Traversal: G,D,H,L,B,E,A,C,I,F,K,J

Inorder Traversal



In-Order Traversal: B,D,A,E,H,G,I,F,C

Inorder Traversal



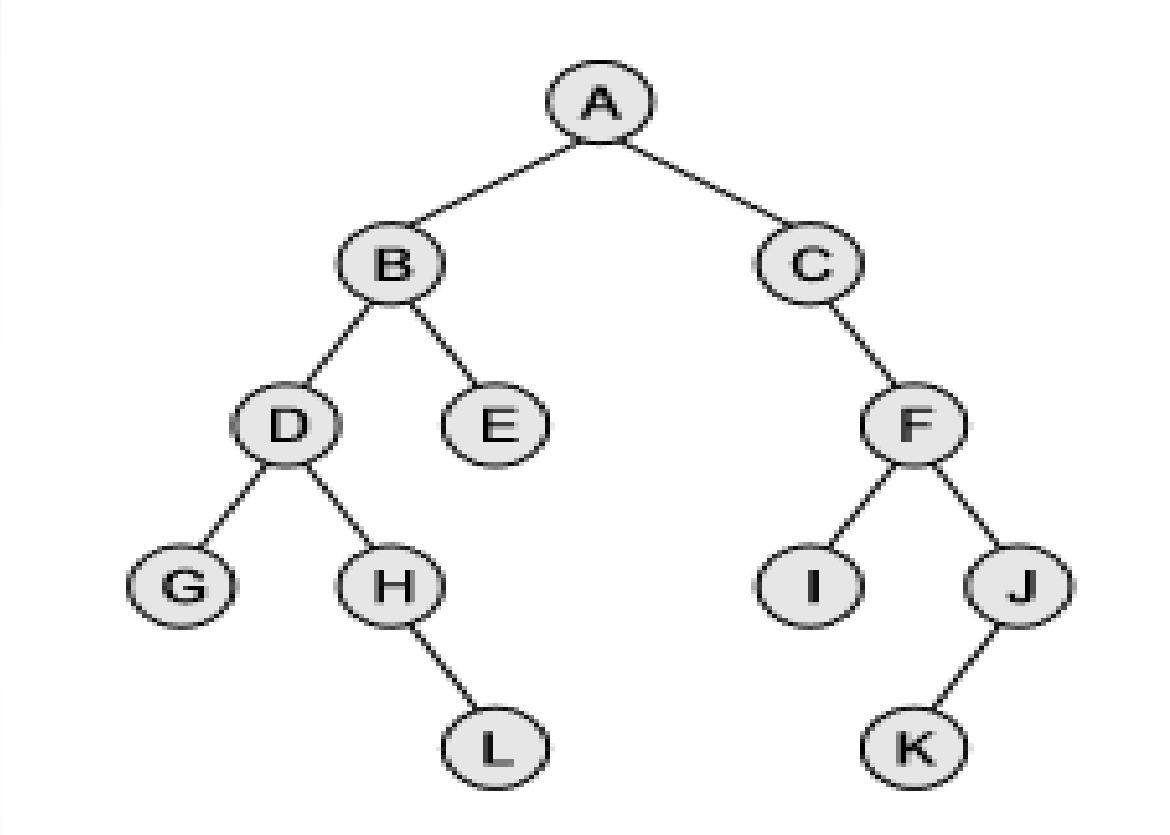
In-Order Traversal: 9, 5, 1, 7, 2, 12, 8, 4, 3, 11

Postorder Traversal

- To traverse a non-empty binary tree in postorder, the following operations are performed recursively at each node.

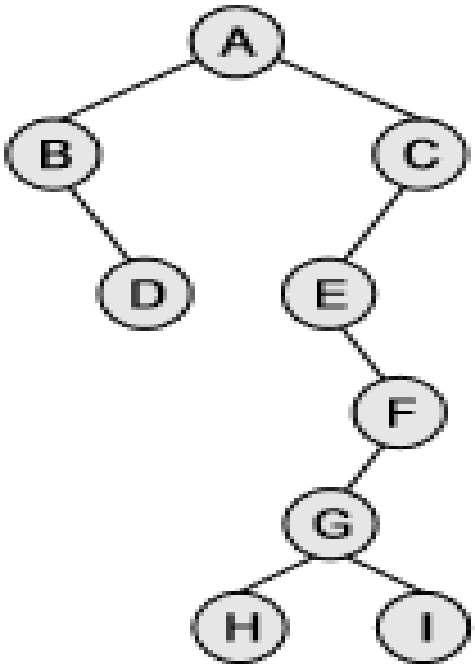
- The traversal starts with the root node of the tree and continues by:
  1. Traversing the left sub-tree.
  2. Traversing the right sub-tree.
  3. Visiting the root node.

Postorder Traversal



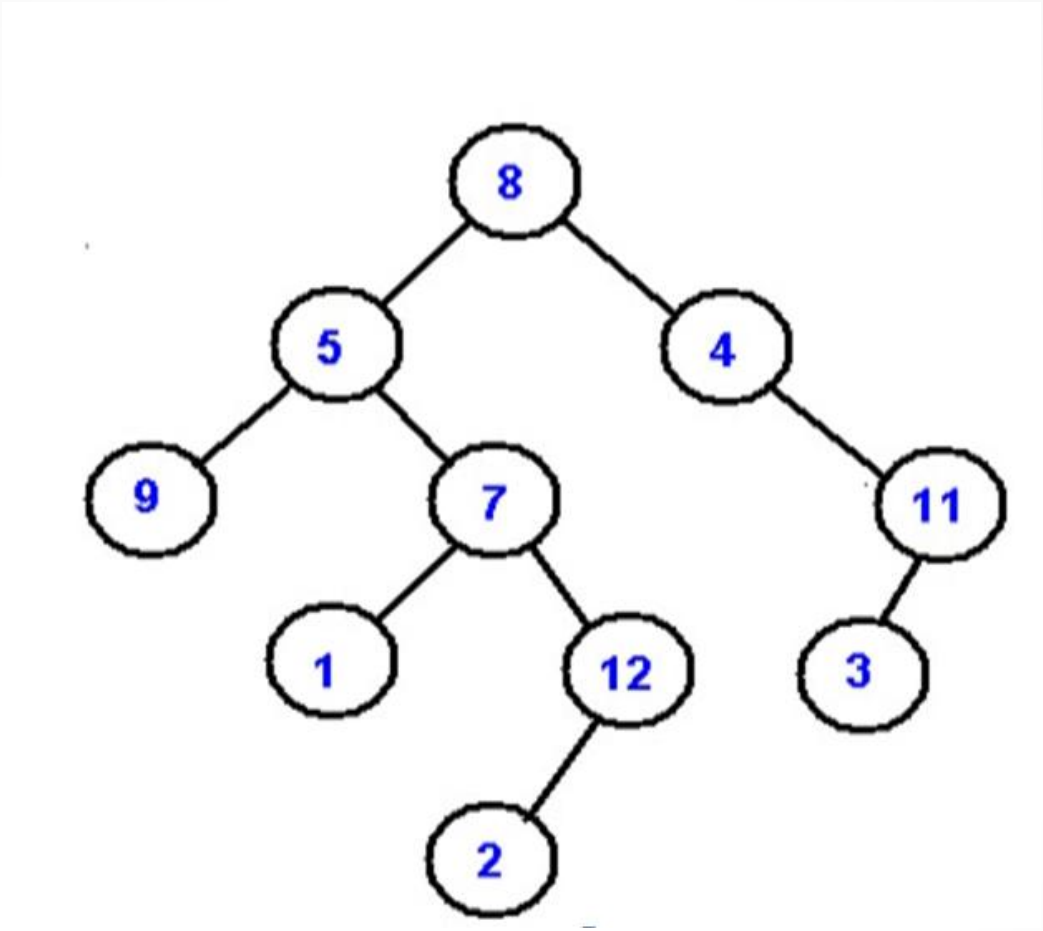
Post-Order Traversal: G,L,H,D,E,B,I,K,J,F,C,A

Postorder Traversal



Post-Order Traversal: D,B,H,I,G,F,E,C,A

Postorder Traversal



Post-Order Traversal: 9, 1, 2, 12, 7, 5, 3, 11, 4, 8

Traversal of Binary Trees

- If the words right and left are interchanged then three new traversals are possible:

- **Converse Preorder Traversal : Root – Right – Left**
- **Converse Inorder Traversal : Right – Root – Left**
- **Converse Postorder Traversal : Right – Left – Root**

Traversal of Binary Trees

- **Example : Find Post-order for given : (REFER CLASS-BOOK)**
- In-order : E A C K F H D B G
- Pre-order : F A E K C D H G B
- **Post-order (Answer) : E C K A H B G D F**

- First See the Preorder. Which element is first in preorder
- Take that element as a ROOT.

- So in this example in Preorder part “F” Becomes Root.
- Because F is the first element in Preorder.

- Now Check in Inorder where F exist in Inorder.

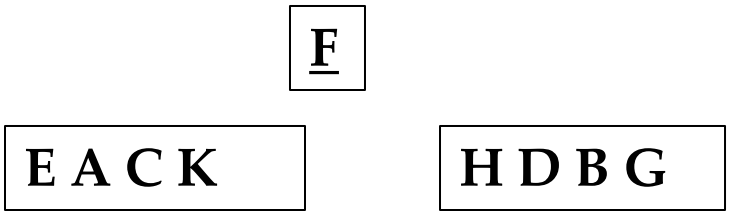
Traversal of Binary Trees

- Inorder = E A C K F H D B G
  Preorder = F A E K C D H G B

- So in Inorder F is there at middle whichever element is
- Of Left side of F becomes Left Element Whichever is at
- Right side becomes right elements.

Traversal of Binary Trees

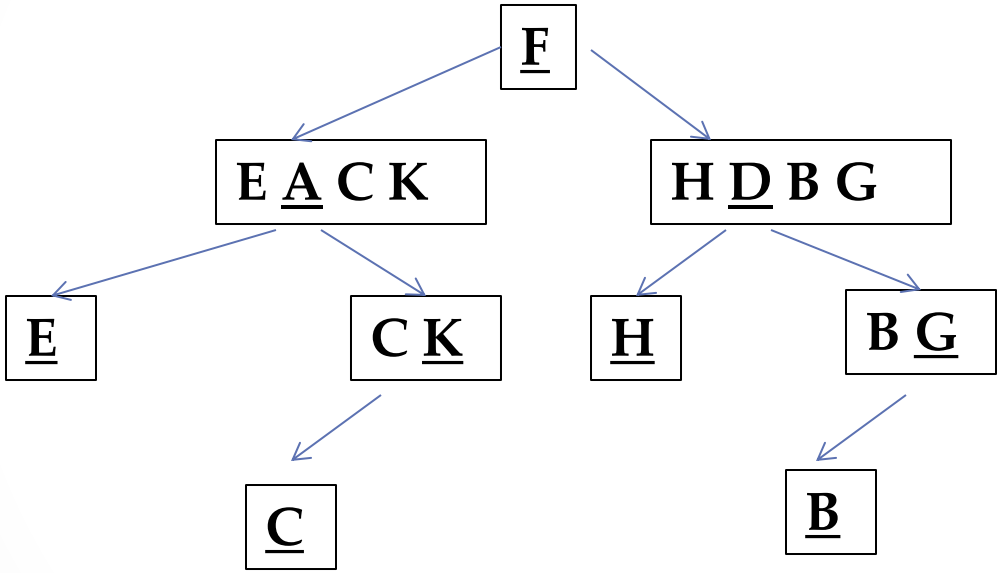
- Inorder = E A C K F H D B G
  Preorder = F A E K C D H G B



- Now Don't see InOrder work for Inorder is over.
- Now check one by one element in Preorder. Whichever element is there after Root that is F is considered as a next root. So Next Element after F is A so now A is become next Root. Whichever element is Left side of A becomes its Left Element whichever is right side of A becomes it Right Element. Now check Next Element that is E then E becomes root. And so on…

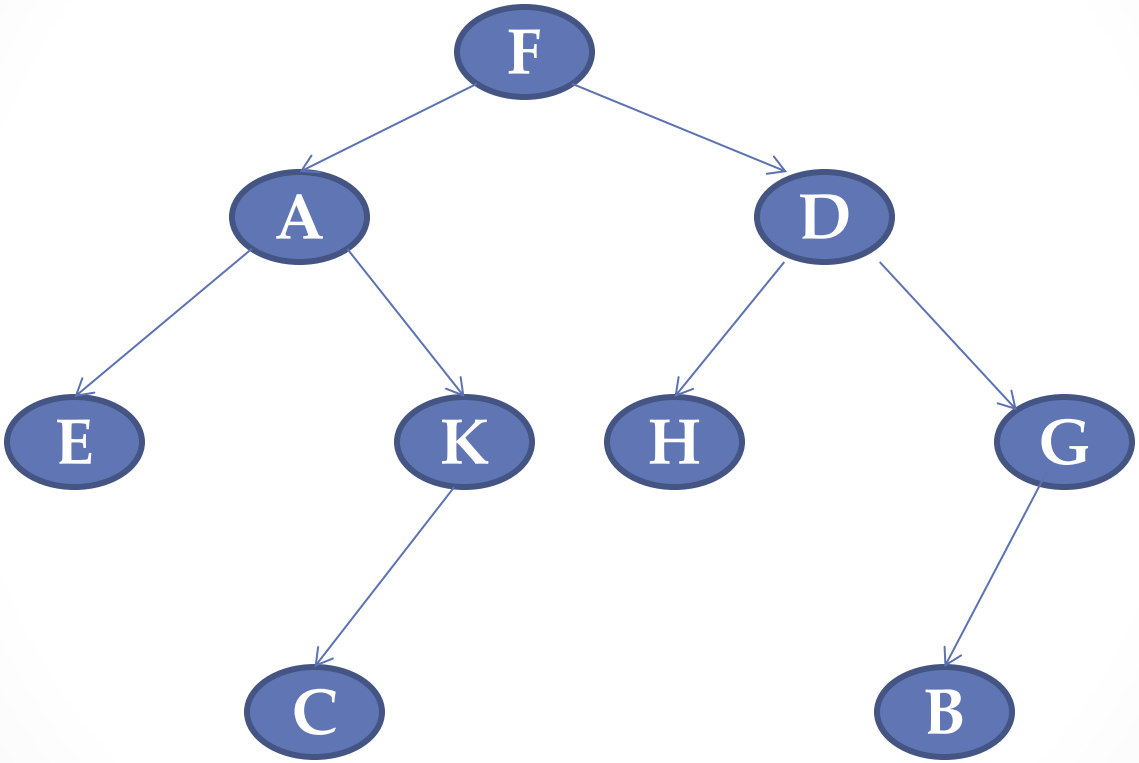
Traversal of Binary Trees

- Inorder = E A C K F H D B G
  Preorder = F A E K C D H G B



Traversal of Binary Trees

Now from above result find out Binary Tree



Traversal of Binary Trees

- Now Find out Post Order from Above Tree..
- That is Left,Right and Root.
- So Post order of above tree is..
- **E C K A H B G D F**

Traversal of Binary Trees

- **Example : Find Post-order for given : (REFER CLASS-BOOK)**
- Pre-order : A B D H E C F G I J
- In-order : D H B E A F C I G J
- **Post-order (Answer) : H D E B F I J G C A**

Traversal of Binary Trees

- **Example : Find Post-order for given : (REFER CLASS-BOOK)**

- Pre-order : 1 2 4 3 5 7 8 6
- In-order : 4 2 1 7 5 8 3 6
- **Post-order (Answer) : 4 2 7 8 5 6 3 1**

Traversal of Binary Trees

- **Example : Find Pre-order for given : (REFER CLASS-BOOK)**
- In-order : D B E A F C G
- Post-order : D E B F G C A

- First See the Postorder. Which element is Last in postorder
- Take that element as a ROOT.

- So in this example in Postorder “A” Becomes Root.
- Because A is the last element in PostOrder.

- Now Check in Inorder where A exist in Inorder.

Traversal of Binary Trees

Inorder = D B E A F C G

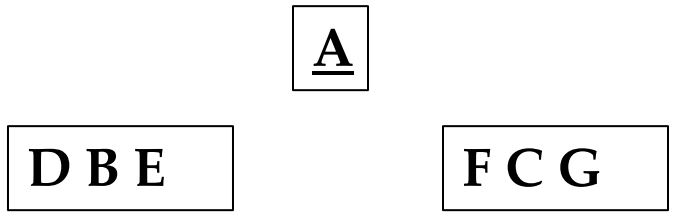
Postorder = D E B F G C A

- So in Inorder A is there at middle whichever element is
- Of Left side of A becomes Left Element Whichever is at
- Right side becomes right elements.

Traversal of Binary Trees

Inorder = D B E A F C G

Postorder = D E B F G C A



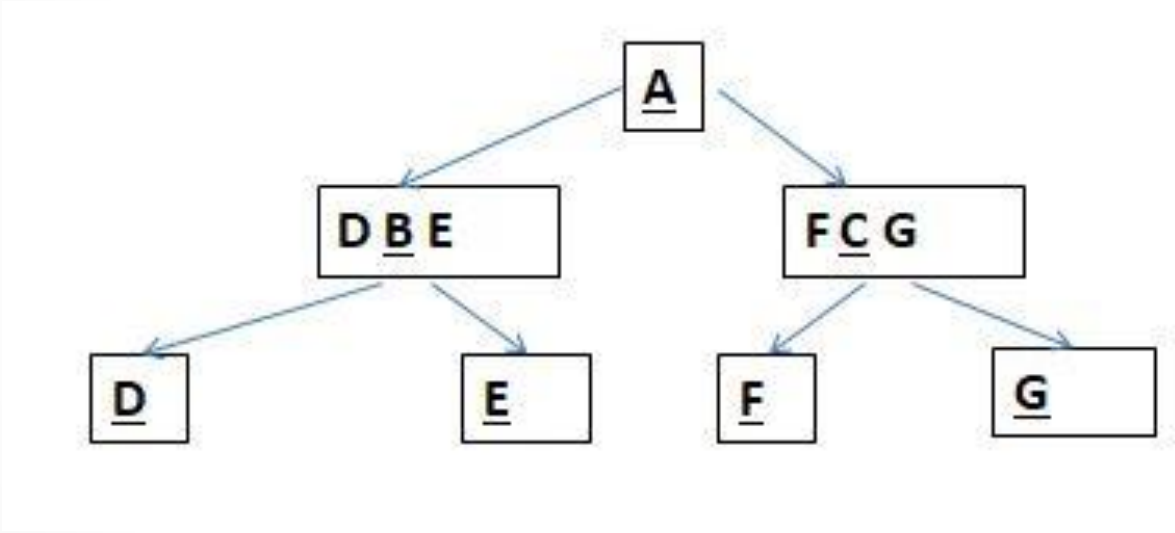
Now Don't see InOrder work for Inorder is over.

Now check one by one element in Post order **from Last**. Whichever element is there before Root that is C is considered as a next root. So Next Element after A is C so now C is become next Root. Whichever element is Left side of C becomes its Left Element whichever is right side of C becomes it Right Element. Now check previous Element that is G then G becomes root. And so on…

Traversal of Binary Trees

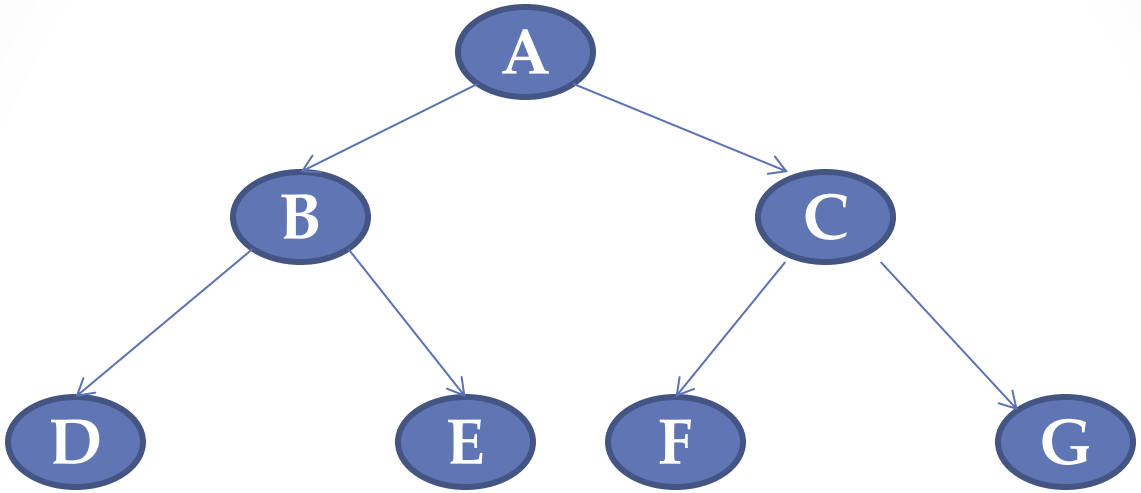
Inorder = D B E A F C G

Postorder = D E B F G C A



Traversal of Binary Trees

Now from above result find out Binary Tree



Now Find out Preorder from Above Tree.. That is Root Left and Right

So Preorder of above tree is..

A B D E C F G

Traversal of Binary Trees

- **Example : Find Pre-order for given : (REFER CLASS-BOOK)**
- Post-order: 4 2 7 8 5 6 3 1
- In-order : 4 2 1 7 5 8 3 6
- **Pre-order (Answer) : 1 2 4 3 5 7 8 6**

Traversal of Binary Trees

- **Example : Find Pre-order for given : (REFER CLASS-BOOK)**
- Post-order: 4 5 2 6 3 1
- In-order : 4 2 5 1 3 6
- **Pre-order (Answer) : 1 2 4 5 3 6**

Traversal of Binary Trees

- **Example : Find Pre-order for given : (REFER CLASS-BOOK)**
- Post-order: 16 21 7 1 9 5 12
- In-order : 16 7 21 12 1 5 9
- **Pre-order (Answer) : 12 7 16 21 5 1 9**

Traversal of Binary Trees

- **Example : Find Pre-order for given : (REFER CLASS-BOOK)**
- Post-order: 9 1 2 12 7 5 3 11 4 8
- In-order : 9 5 1 7 2 12 8 4 3 11
- **Pre-order (Answer) : 8 5 9 7 1 12 2 4 11 3**

Conversion of General Tree to Binary Tree

- A general tree can have any number of nodes.
- To represent general trees as binary trees , two pointers are maintained. One pointer points to the first child of the node and the other pointer points to the siblings of the node.
- The structure of the node in the general tree can be defined as:

```
struct gen
{
        int data;
        struct gen *fchild;
        struct gen *sibling;
};
```

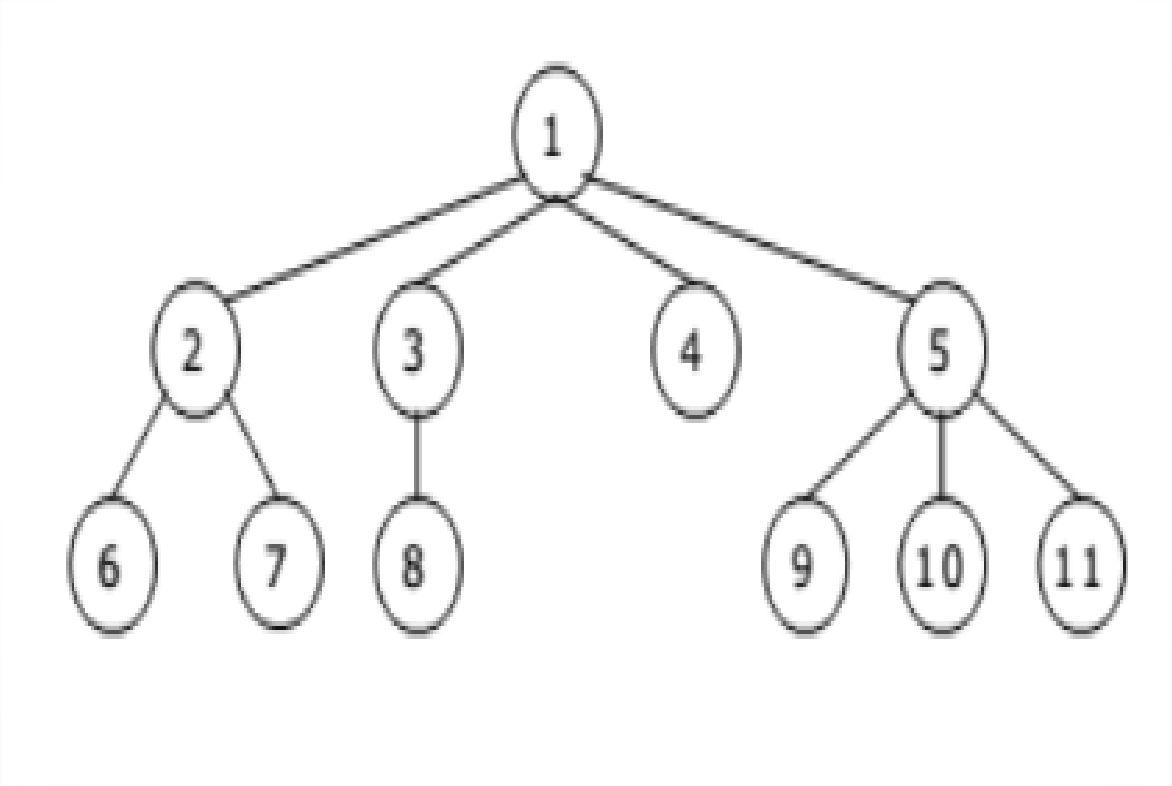
Conversion of General Tree to Binary Tree

- Following are the steps of converting given General tree into Binary tree:

- As a first step we delete all the branches originating in every node except the left most branch.
- We draw edges from a node to the node on the right, if any, which is situated at the same level.
- Once it is done, then for any node we choose its left and right children in the following manner:
  - The left child id the node, which is immediately below the given node, and the right child is the node to the immediate right of the given node on the same horizontal line.

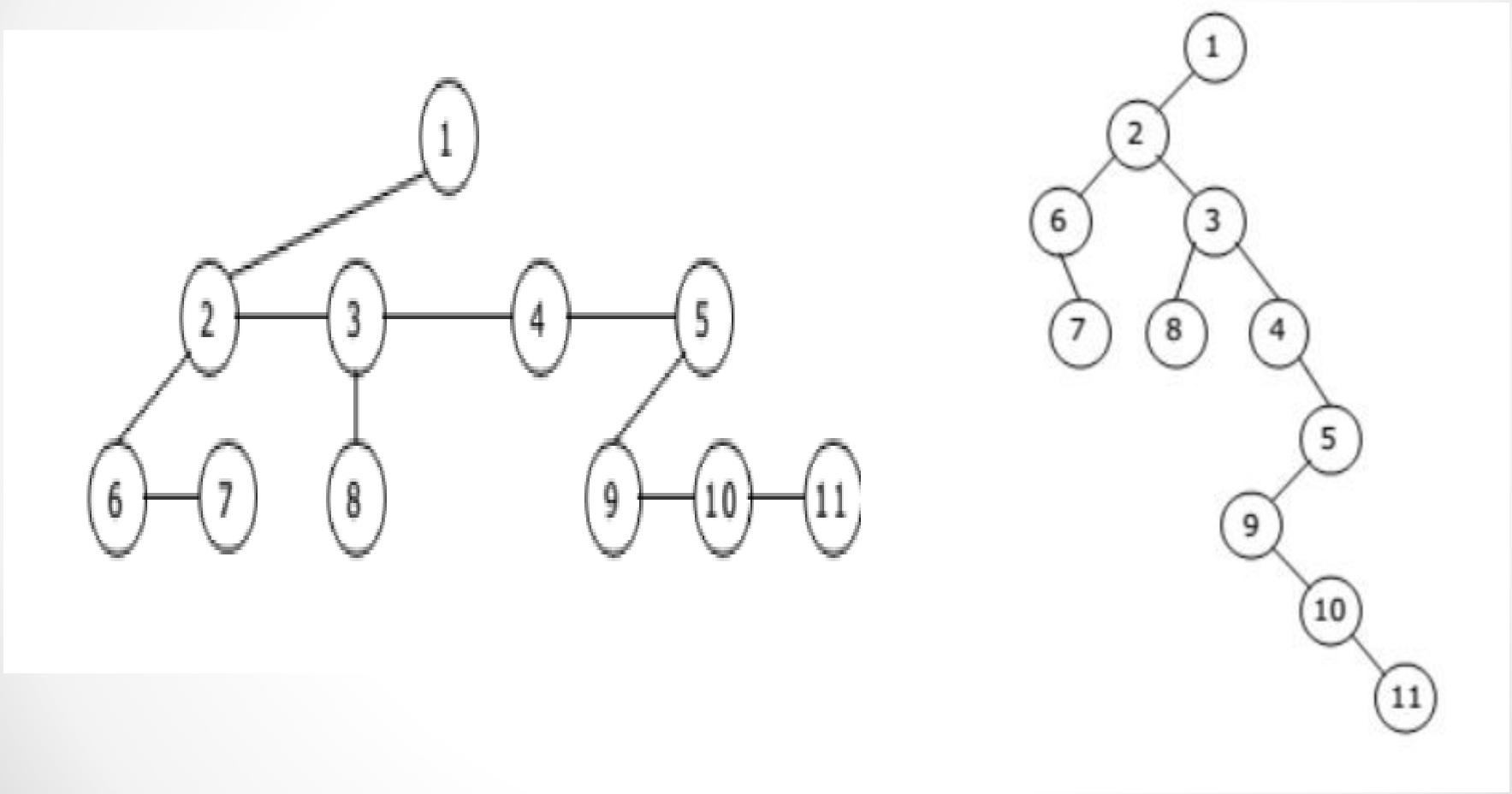
Conversion of General Tree to Binary Tree

- Example-1: General Tree given:



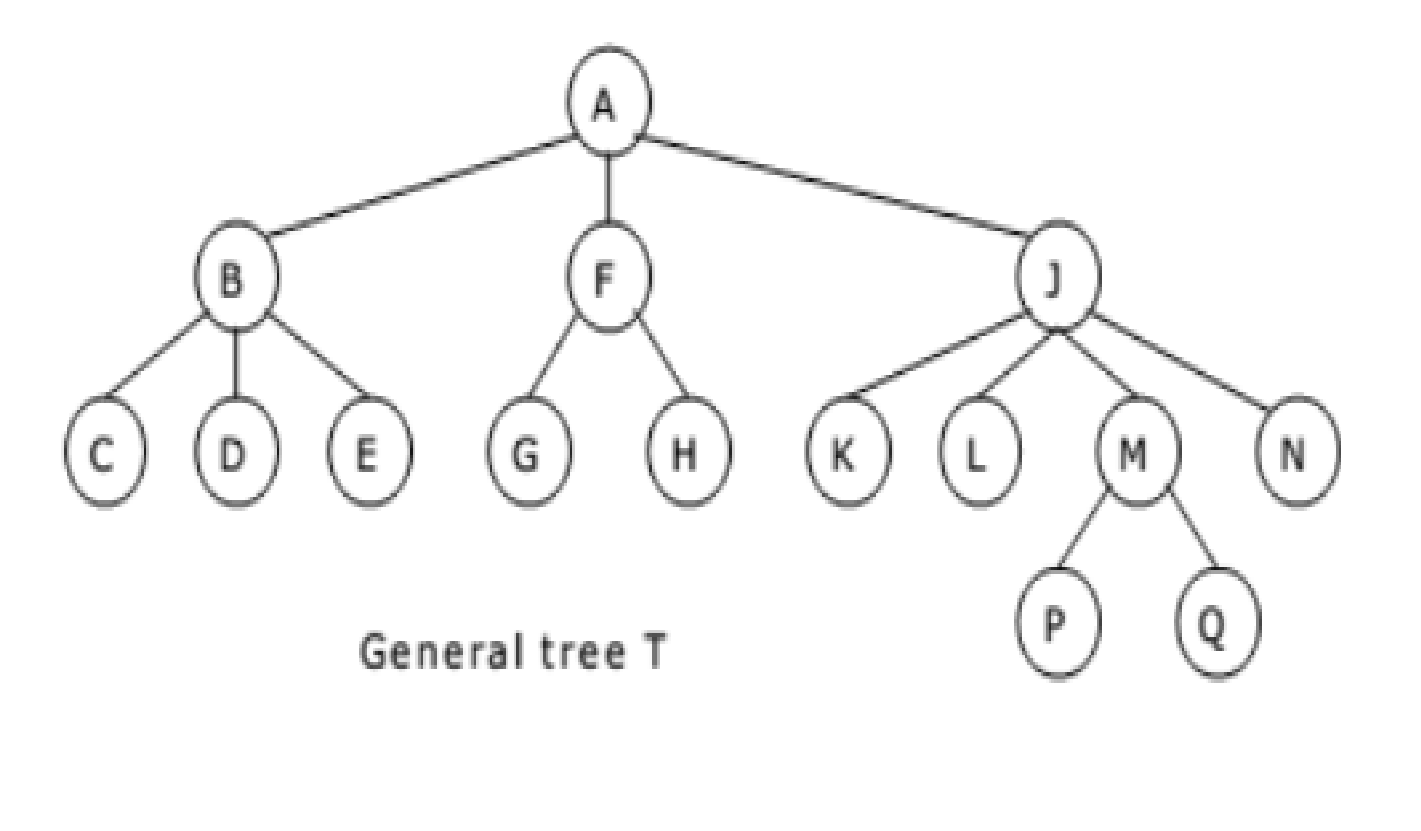
Conversion of General Tree to Binary Tree

- Example-1: Binary Tree of the given General Tree



Conversion of General Tree to Binary Tree

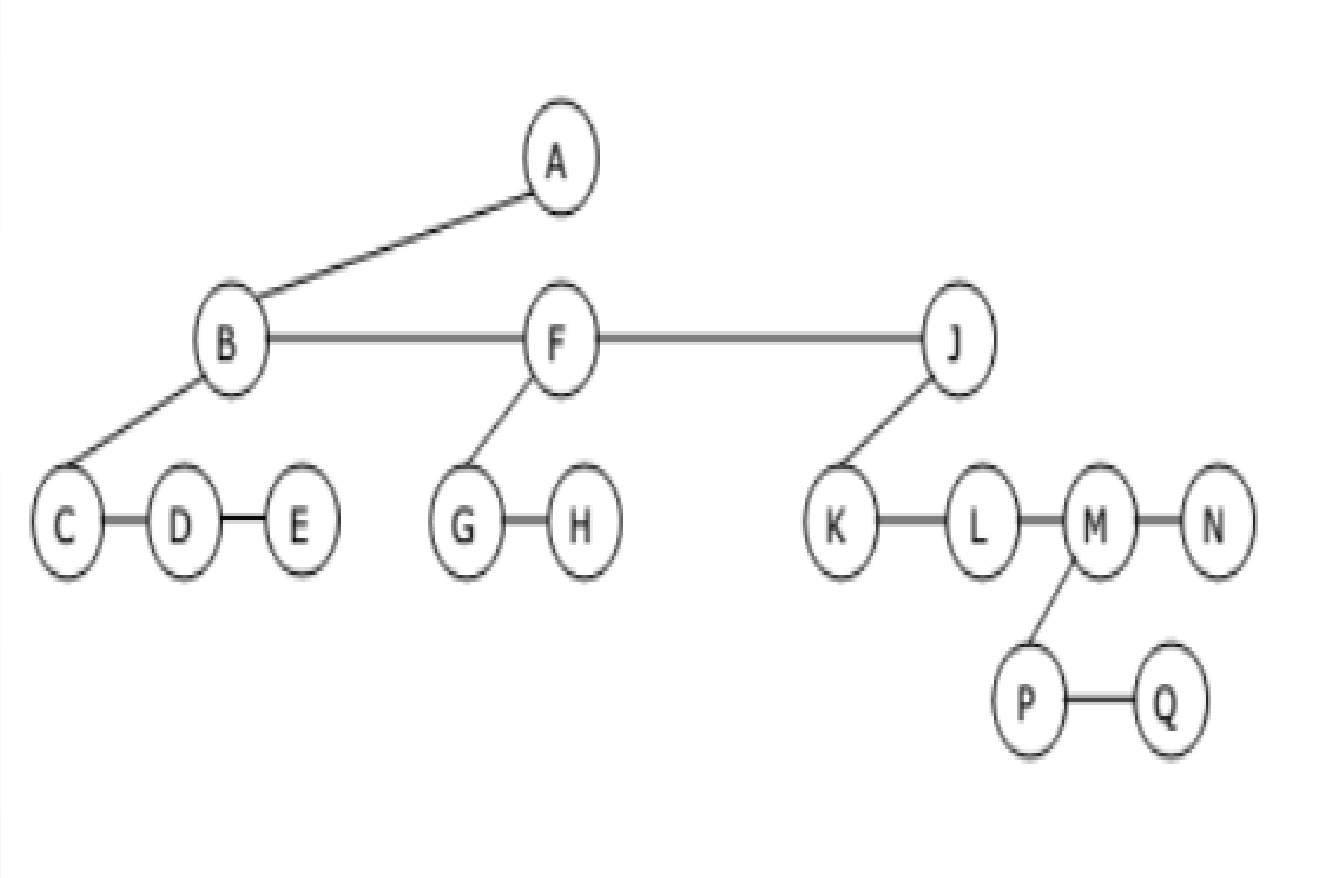
- Example-2: General Tree given:



General tree T

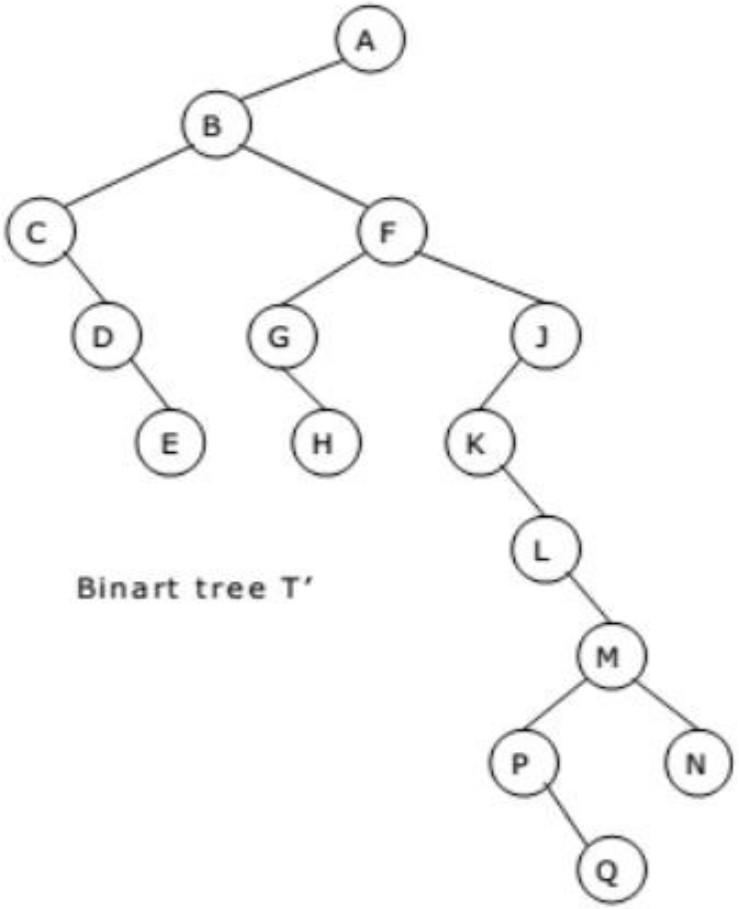
Conversion of General Tree to Binary Tree

- Example-2 : Binary Tree of the given General Tree



Conversion of General Tree to Binary Tree

- Example-2 : Binary Tree of the given General Tree

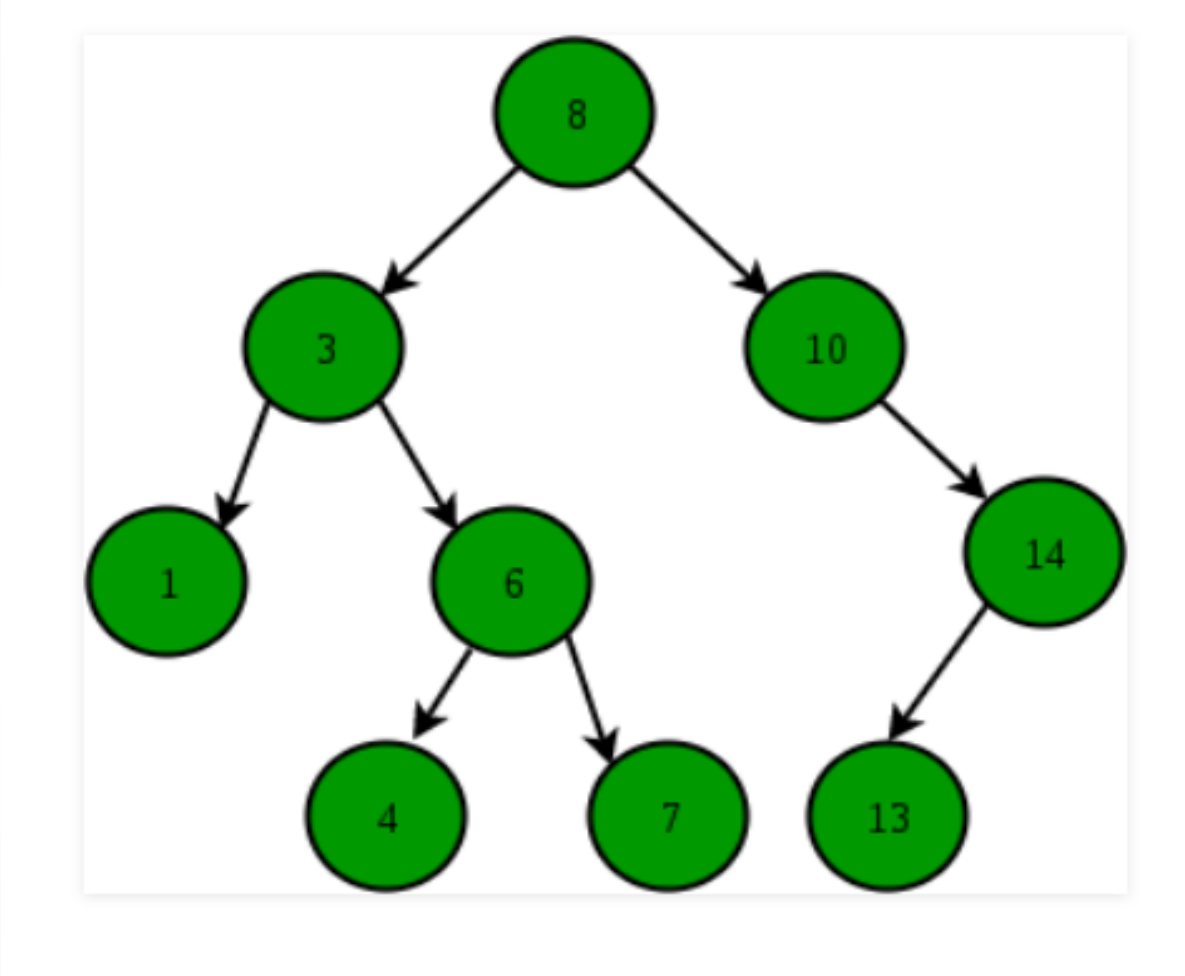


Binart tree T'

Binary Search Tree

- Binary Search Tree is a node-based binary tree data structure which has the following properties:
  - The left subtree of a node contains only nodes with keys lesser than the node's key.
  - The right subtree of a node contains only nodes with keys greater than the node's key.
  - The left and right subtree each must also be a binary search tree.

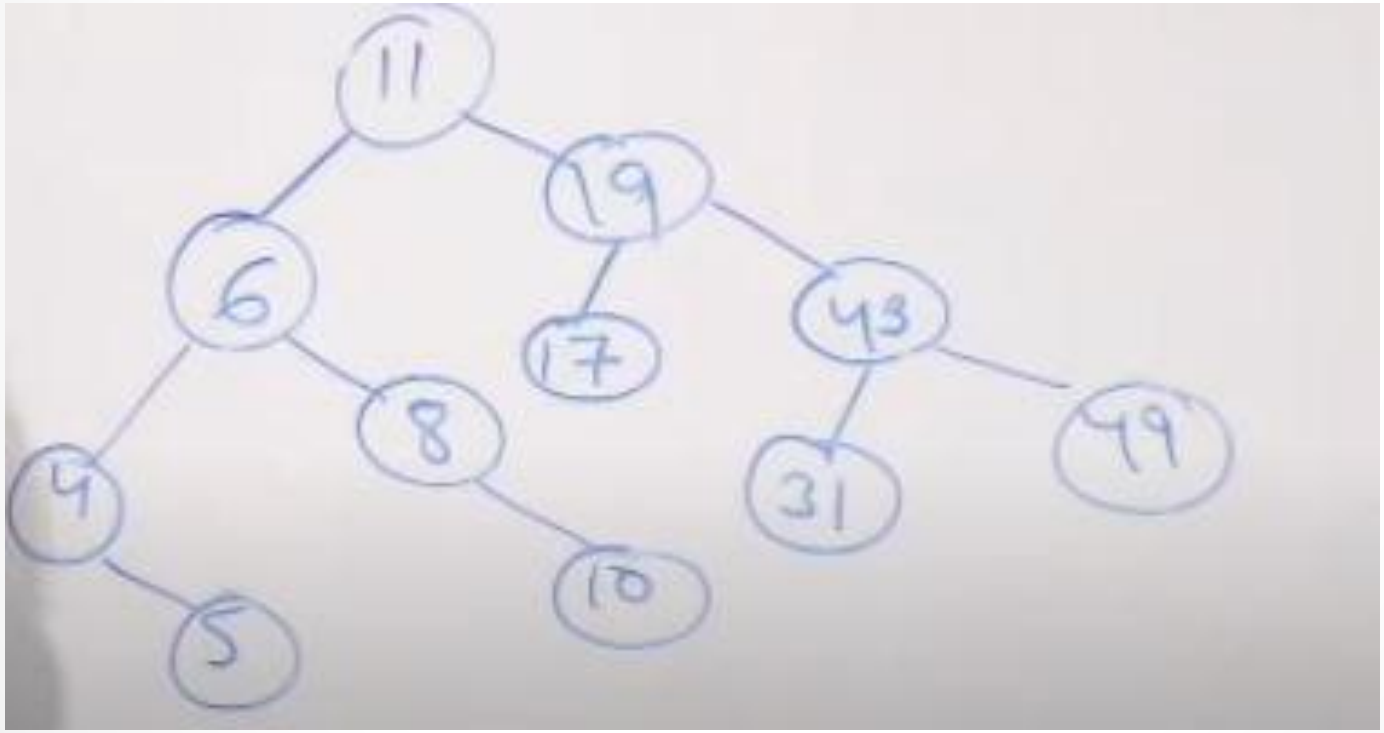
Binary Search Tree



Binary Search Tree

- Elements are

11,6,8,19,4,10,5,17,43,49,31



Binary Search Tree

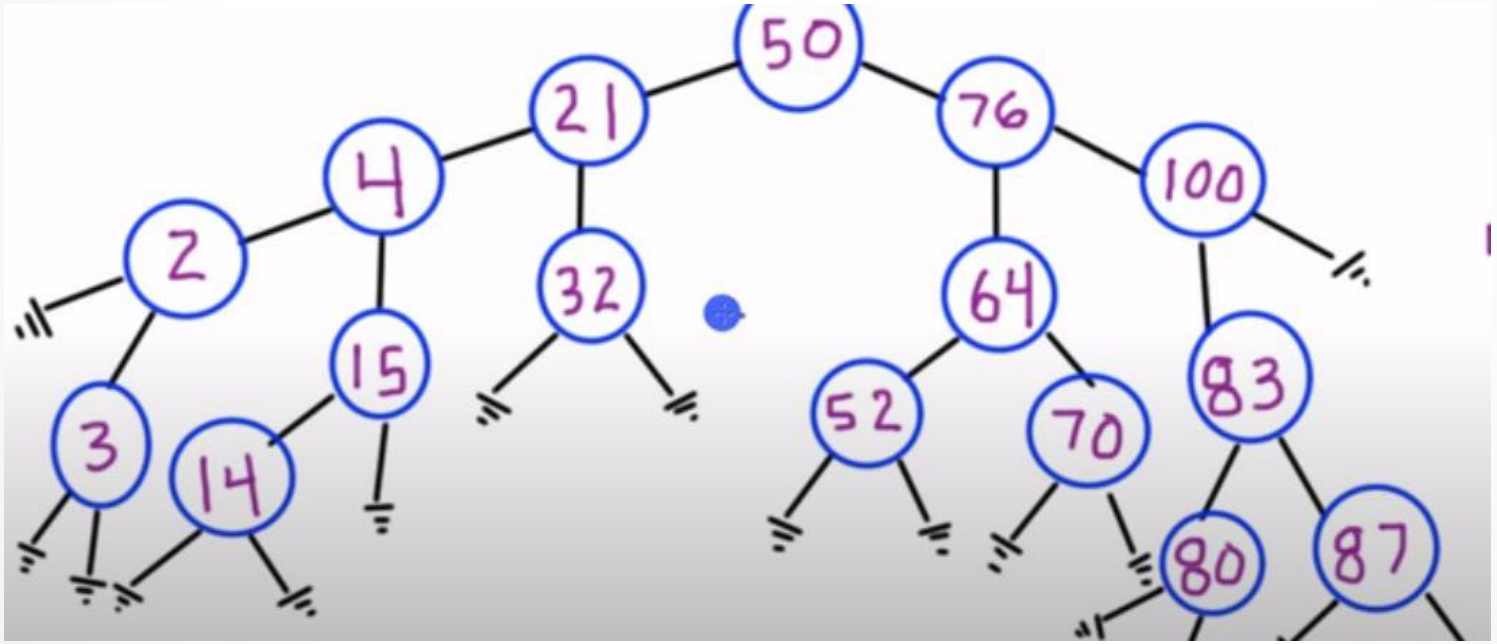
- Elements are

6, 5, 9, 10, 11, 13, 4, 3, 7, 12

Binary Search Tree

- Elements are

50,76,21,4,32,64,15,52,14,100,83,2,3,70,87,80



Thank You…!!!!